CAHIER DES CHARGES

SAE 3.02

Application de simulation et de gestion de carrefours communicants

Formation	BUT Réseaux & Télécommunications
Groupe	Yoda
Étudiants	Jules LANDAUER et Mael SASORITH
Année universitaire	2026–2027

1. Présentation du projet

1.1 Contexte

Dans le cadre de la SAE 3.02 du BUT Réseaux & Télécommunications, nous devons développer une application en Python utilisant notamment une interface graphique et des communications réseau.

Nous avons choisi de réaliser une mini-simulation d'un carrefour connecté. Le but est de représenter simplement un carrefour avec quatre feux de circulation et plusieurs véhicules.

Le projet a été volontairement limité afin de rester réalisable dans le temps disponible et de conserver un programme que nous pouvons comprendre et expliquer.

1.2 Problématique

Comment réaliser une petite simulation de carrefour permettant de représenter des véhicules et des feux de circulation, tout en mettant en œuvre une interface graphique, une communication réseau et une base de données ?

1.3 Solution proposée

Nous proposons de développer une application en Python avec PyQt6.

L'application permettra :

- D'afficher un carrefour ;
- D'afficher quatre feux ;
- D'ajouter des véhicules ;
- De faire déplacer les véhicules ;
- De démarrer et arrêter la simulation ;
- De faire changer automatiquement les feux ;
- D'envoyer des messages par TCP ;
- D'utiliser JSON pour les messages ;
- D'enregistrer certains événements dans SQLite.

La partie simulation sera séparée de la partie affichage afin de garder une organisation simple.

2. Objectifs du projet

Objectif principal

Réaliser une petite simulation graphique d'un carrefour connecté en Python.

Objectifs secondaires

- Créer une interface avec PyQt6 ;
- Représenter un carrefour ;

- Gérer quatre feux ;
- Créer et déplacer des véhicules ;
- Démarrer et arrêter la simulation ;
- Mettre en place un serveur TCP local ;
- Envoyer des messages JSON ;
- Utiliser un `QThread` pour la communication réseau ;
- Enregistrer les événements dans SQLite ;
- Tester l'application.

3. Périmètre fonctionnel

3.1 Interface graphique

L'application devra proposer une interface permettant à l'utilisateur de visualiser les éléments principaux de la simulation et d'accéder aux commandes nécessaires.

- Lancer et arrêter la simulation ;
- Visualiser le carrefour ;
- Visualiser les véhicules ;
- Visualiser l'état des feux ;
- Accéder aux informations importantes de la simulation ;

3.2 Simulation

La simulation représente un seul carrefour.

Elle contient :

- Quatre feux ;
- Plusieurs véhicules ;
- Quatre directions possibles ;
- Une évolution de la position des véhicules ;
- Un changement automatique des feux.

La simulation est gérée séparément de l'affichage graphique.

3.3 Gestion des véhicules

Chaque véhicule possède :

- Un identifiant ;

- Une position x et y ;
- Une direction ;
- Une vitesse simple.

Les véhicules sont ajoutés depuis l'interface et se déplacent automatiquement lorsqu'une simulation est en cours.

3.4 Gestion des feux

Le carrefour possède quatre feux : Nord, Sud, Est et Ouest.

Chaque feu peut avoir trois états :

- vert ;
- orange ;
- rouge ;

Les changements d'état sont gérés automatiquement par la simulation.

3.5 Communications V2X / V2I

Une communication TCP locale est utilisée pour montrer l'échange d'informations entre la simulation et un serveur.

Le serveur fonctionne sur : 127.0.0.1:5000

Lorsqu'un véhicule est ajouté, un message JSON est envoyé au serveur.

Exemple de message possible :

```
{
    "type": "vehicle",
    "id": 1,
    "speed": 2
}
```

3.6 Gestion des erreurs

L'application devra gérer les principales erreurs pouvant apparaître pendant son fonctionnement, notamment :

- Connexion impossible ;
- Serveur indisponible ;
- Déconnexion d'un client ;
- Message reçu incorrect ;
- Données manquantes ou mal formées ;
- Erreur de communication.

4. Spécifications techniques

4.1 Langage

Le projet sera développé exclusivement en Python 3.13.5.

4.2 Interface graphique

L'interface sera développée avec PyQt6. Les modules autorisés sont notamment :

- PyQt6.QtWidgets
- PyQt6.QtCore
- PyQt6.QtGui

Pour la partie graphique du carrefour, QGraphicsScene et QGraphicsView seront privilégiés.

4.3 Concurrence

La communication réseau est réalisée dans un `QThread` afin d'éviter de bloquer l'interface graphique.

Le principe est :

Interface → QThread → Client TCP → Serveur TCP

Le résultat de la communication est transmis à l'interface avec un signal Qt.

Le thread réseau ne modifie pas directement les éléments graphiques.

4.4 Base de données

SQLite pourra être utilisée pour conserver les informations utiles à la simulation, notamment les historiques, les métriques, les événements ou les statistiques.

4.5 Environnement de développement

Le projet sera développé dans un environnement virtuel Python afin d'isoler ses dépendances.

```
python -m venv .venv

pip install -r requirements.txt
```

Le code sera versionné avec Git et hébergé sur un dépôt GitHub public.

5. Architecture technique

L'organisation du projet va suivre une architecture modulaire de ce type :

```
SAE-3.02/
├── README.md
├── requirements.txt
├── .gitignore
├── src/
│   ├── main.py
│   ├── gui.py
│   └── network_worker.py
│   ├── client.py
```

```
│   ├── server.py
│   ├── simulation.py
│   ├── vehicle.py
│   └── traffic_light.py
│   └── database.py

└── database/
│       └── traffic.db
└── docs/
    ├── Cahier_des_Charges.pdf
    ├── Bilan_Projet.pdf
    └── IAGraphie.pdf
```

Nous avons choisi une architecture simple avec plusieurs fichiers ayant chacun un rôle précis. La simulation est séparée de l'interface afin de faciliter le développement et les tests.

6. Contraintes techniques

- Utilisation obligatoire de Python 3.15 ;
- Utilisation de PyQt6 pour l'interface graphique ;
- Utilisation de sockets Python pour les communications ;
- Absence de framework réseau haut niveau ;
- Utilisation des signaux et slots pour les échanges entre threads et interface ;
- Présence d'un fichier requirements.txt avec les versions exactes des dépendances externes ;
- Présence d'un README.md à la racine du dépôt ;
- Utilisation de Git et GitHub ;

7. Périmètre fonctionnel et coût humain

Les estimations ci-dessous sont prévisionnelles et sont ajustées selon l'avancement réel du projet.

Fonctionnalité	Priorité	Complexité	Charge estimée
Interface graphique principale	1	3/5	4 h
Affichage du carrefour	1	4/5	5 h
Gestion des feux	1	3/5	3 h
Gestion des véhicules	1	4/5	5 h
Simulation du trafic	1	4/5	5 h
Gestion des threads	1	4/5	4 h
Gestion des erreurs réseau	1	3/5	2 h

Enregistrement des données	2	3/5	3 h
Affichage de statistiques	2	3/5	3 h
Fonctionnalités non prévues	4	—	—

Charge totale prévisionnelle : environ 34 heures pour le binôme.

8. Organisation du travail

Le projet sera réalisé en binôme par Jules Landauer et Mael Sasorith. La répartition est prévue de manière équilibrée afin que chacun participe au développement, aux tests et à la documentation.

8.1 Jules LANDAUER

- main.py
- gui.py
- Affichage du carrefour ;
- Affichage des véhicules ;
- Boutons ;
- Intégration de l'interface ;
- Tests interface ;
- README.

8.2 Mael SASORITH

- simulation.py
- vehicle.py
- traffic_light.py
- server.py
- client.py
- network_worker.py
- database.py
- tests réseau et SQLite.

8.3 Travail commun

- conception ;
- intégration ;

- corrections ;
- tests finaux ;
- GitHub ;
- documentation finale.

9. Planning prévisionnel

Les temps seront comparés avec les temps réellement réalisés grâce aux relevés de décisions.

Étapes	Temps
Conception	2 h
Interface	5 h
Simulation	7 h
Réseau	6 h
QThread	4 h
SQLite	2 h
Tests et corrections	5 h
Documentation	2 h
Intégration finale	3 h
GitHub / finalisation	2 h
Total	38 h

10. Livrables attendus

Le livrable final prendra la forme d'un dépôt GitHub public contenant l'ensemble des éléments demandés.

Code source Python : Tous les scripts .py nécessaires au fonctionnement de l'application, organisés de manière modulaire.

README.md : Présentation du projet, des membres du groupe, des prérequis, de l'installation, de la configuration et de l'utilisation.

requirements.txt : Liste des versions exactes des bibliothèques externes requises.

Cahier des charges : Présentation du contexte, des besoins, des spécifications, des fonctionnalités, de l'organisation et du planning.

Bilan_Projet.pdf : Bilan des fonctionnalités, difficultés techniques, organisation et comparaison entre le planning prévu et le temps réel.

IAGraphie.pdf : Présentation de l'utilisation éventuelle d'outils d'IA, exemples de prompts et réflexion sur cet usage.

11. Tests et validation

Des tests seront réalisés pendant le développement puis lors d'une phase finale de validation.

- Lancement de l'application ;
- Affichage de l'interface ;
- Fonctionnement de la simulation ;
- Changement d'état des feux ;
- Déplacement des véhicules ;
- Envoi et réception des données réseau ;
- Gestion d'une perte de connexion ;
- Gestion des erreurs de communication ;
- Synchronisation entre les threads ;
- Enregistrement des données.

12. Suivi du projet

Un relevé de décisions sera réalisé chaque semaine. Il permettra de suivre le temps réellement consacré au projet et la répartition du travail entre Jules et Mael.

- Tâches réalisées par chaque étudiant ;
- Problèmes rencontrés ;
- Décisions prises ;
- Objectifs de la semaine suivante.

14. Conclusion

Ce cahier des charges présente les principaux besoins fonctionnels et techniques retenus pour la SAE 3.02. L'objectif est de réaliser une application Python permettant de représenter et de simuler un environnement de carrefour connecté tout en mettant en pratique les notions de programmation graphique, de communication réseau et de programmation concurrente.

Le projet sera développé progressivement par Jules Landauer et Mael, avec une répartition équilibrée des tâches. Le suivi hebdomadaire permettra de comparer les objectifs prévus avec le travail réellement réalisé.